EE161 Homework 4
Processor Control and Datapath

Kaushik Vada

November 4, 2025

Question 1: Logic Path and Delay

The single-cycle datapath from the handout provides the following component delays: instruction and data memories 200 ns, the main ALU 20 ns, each adder (for $PC + 4$ and the branch target) 10 ns, every multiplexer 5 ns, the sign-extension and shift-left-two blocks 5 ns each, and register-file reads or writes 5 ns. These values are used to accumulate the latency of each instruction.

Part A

$$
\begin{aligned}
t_{\text{addi}} &= T_{\text{IM}} + T_{\text{RegR}} + T_{\text{SignExt}} + T_{\text{ALUSrc MUX}} + T_{\text{ALU}} + T_{\text{WB MUX}} + T_{\text{RegW}} \\
&= 200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 20\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} = 245\,\text{ns}, \\
t_{\text{add}} &= T_{\text{IM}} + T_{\text{RegR}} + T_{\text{ALUSrc MUX}} + T_{\text{ALU}} + T_{\text{WB MUX}} + T_{\text{RegW}} \\
&= 200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 20\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} = 240\,\text{ns}, \\
t_{\text{beq}} &= \max\big(T_{\text{IM}} + T_{\text{RegR}} + T_{\text{ALUSrc MUX}} + T_{\text{ALU}},\, T_{\text{IM}} + T_{\text{SignExt}} + T_{\text{ShiftL2}} + T_{\text{Branch Adder}}\big) + T_{\text{PC MUX}} \\
&= \max\big(200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 20\,\text{ns},\, 200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 10\,\text{ns}\big) + 5\,\text{ns} \\
&= 235\,\text{ns}, \\
t_{\text{lw}} &= T_{\text{IM}} + T_{\text{RegR}} + T_{\text{SignExt}} + T_{\text{ALUSrc MUX}} + T_{\text{ALU}} + T_{\text{DM}} + T_{\text{WB MUX}} + T_{\text{RegW}} \\
&= 200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} + 20\,\text{ns} + 200\,\text{ns} + 5\,\text{ns} + 5\,\text{ns} = 445\,\text{ns}.
\end{aligned}
$$

Part B

The slowest instruction in this set is the load word:

$$t_{\text{slowest}} = t_{\text{lw}} = 445\,\text{ns}, \qquad f_{\max} = \frac{1}{t_{\text{slowest}}} \approx 2.25\,\text{MHz}.$$

Question 2: Control Signals for `SW`

(A) RegWrite: 0. A store does not write a register, so the register-file write port must remain disabled.

(B) ALUSrc: 1. The base register is added to the sign-extended immediate to form the byte address, so the ALU must take the immediate on its second input.

(C) ALUOp: 00 (the "add" operation). Both loads and stores use the ALU to compute an address via addition, so the control unit drives the add code for the ALU.

(D) MemWrite: 1. The store instruction writes the data from register `rt` into memory, so the data-memory write enable must be asserted.

Question 3: Adding an Unconditional Jump

To support the jump instruction `j`, extend the datapath as follows:

- Form the 32-bit jump target by concatenating the upper four bits of the incremented program counter with the 26-bit immediate shifted left by two: $\{\, PC{+}4[31{:}28],\ \text{instr}[25{:}0] \ll 2 \,\}$. This reuses the existing $PC+4$ adder and introduces a shift-left-two block on the 26-bit immediate.
- Insert a new 2:1 multiplexer on the PC input that selects between the existing next-PC value (either $PC + 4$ or the branch target) and the jump target. Its select line is driven by a new control signal `Jump`.
- Extend the main controller so that `Jump` is asserted when the opcode corresponds to a J-type instruction and deasserted otherwise. When `Jump`=1 the PC multiplexer forwards the jump target and suppresses branch decisions; all other control signals remain as in the existing single-cycle implementation.

These additions enable the processor to fetch the instruction stream sequentially, take conditional branches, or perform absolute jumps based solely on the opcode.